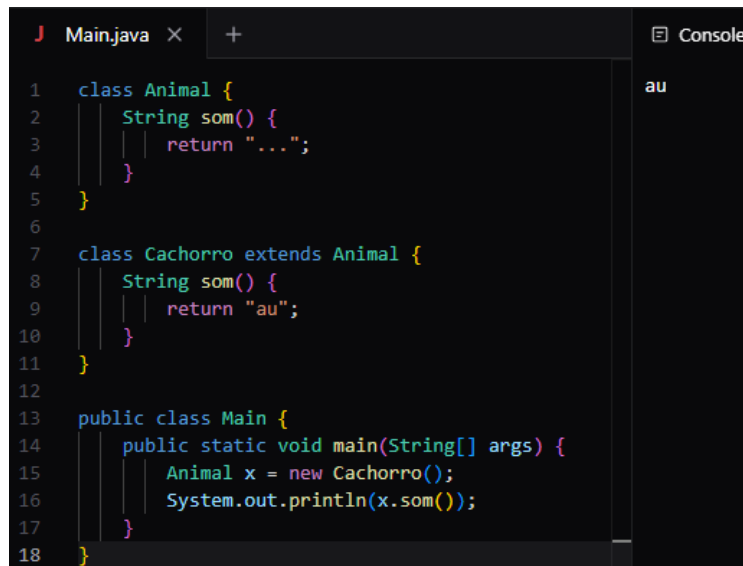


1. Java

a) Qual é a saída?



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Animal' with a method 'som()' that returns '...'. A subclass 'Cachorro' extends 'Animal' and overrides 'som()' to return 'au'. The 'Main' class creates an instance of 'Cachorro' and calls its 'som()' method. The console output on the right shows 'au'.

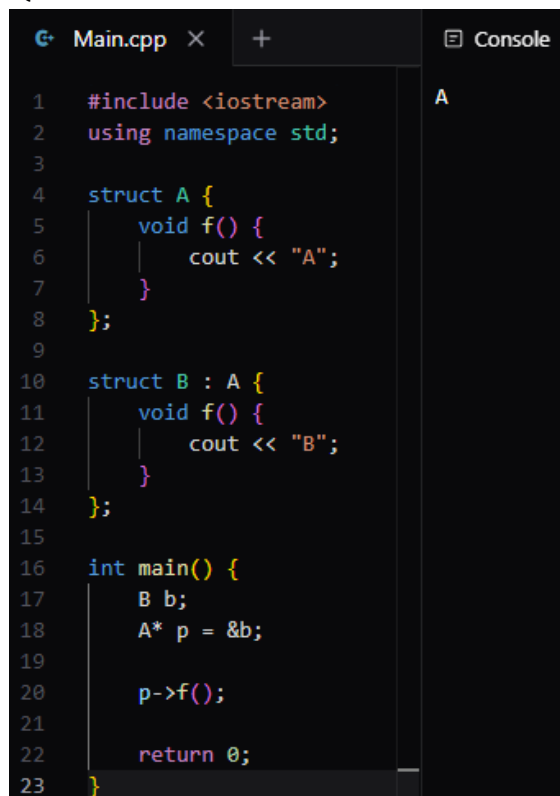
```
1 class Animal {
2     String som() {
3         return "...";
4     }
5 }
6
7 class Cachorro extends Animal {
8     String som() {
9         return "au";
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         Animal x = new Cachorro();
16         System.out.println(x.som());
17     }
18 }
```

Console: au

b) A decisão foi tomada na compilação ou na execução?
Na execução pois som() é um método sobrescrito.

2. C++

a) Qual é a saída?



The screenshot shows a C++ IDE with a file named 'Main.cpp'. The code defines a base struct 'A' with a method 'f()' that prints 'A'. A derived struct 'B' inherits from 'A' and overrides 'f()' to print 'B'. The 'main' function creates a 'B' object, takes its address, and calls 'f()' through a pointer of type 'A*'. The console output on the right shows 'A'.

```
1 #include <iostream>
2 using namespace std;
3
4 struct A {
5     void f() {
6         cout << "A";
7     }
8 };
9
10 struct B : A {
11     void f() {
12         cout << "B";
13     }
14 };
15
16 int main() {
17     B b;
18     A* p = &b;
19
20     p->f();
21
22     return 0;
23 }
```

Console: A

b) A decisão foi tomada na compilação ou na execução?
Na compilação pois o ponto importante é que f() não é virtual.

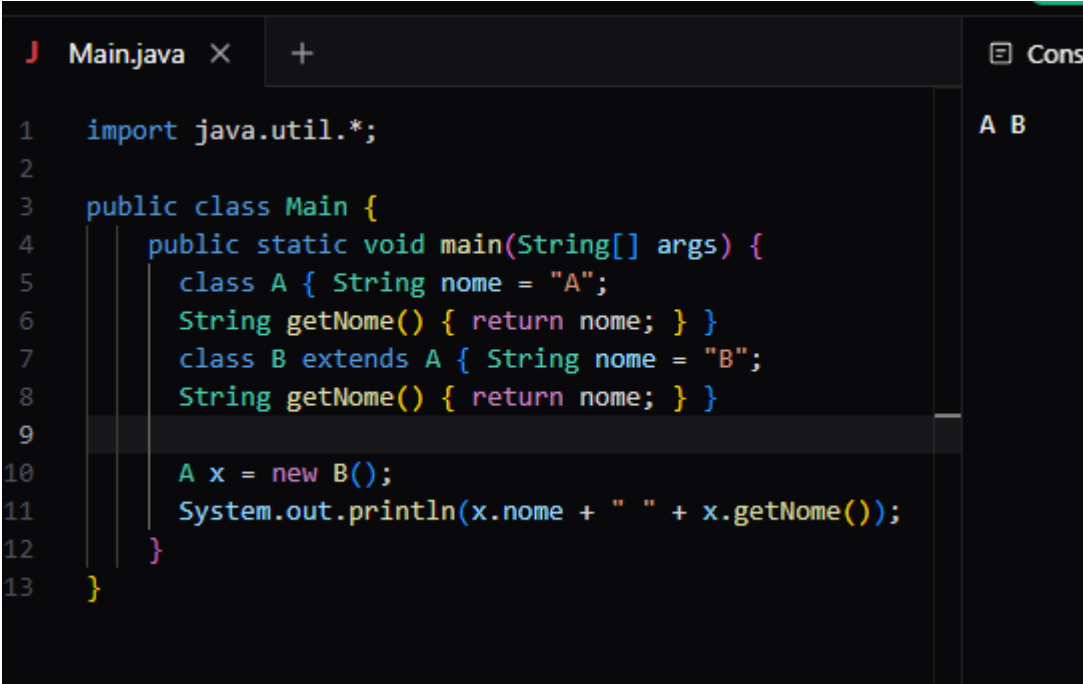
Exercício qual método é chamado

Página 22

3. Java

- (a) Qual é a saída?

A B



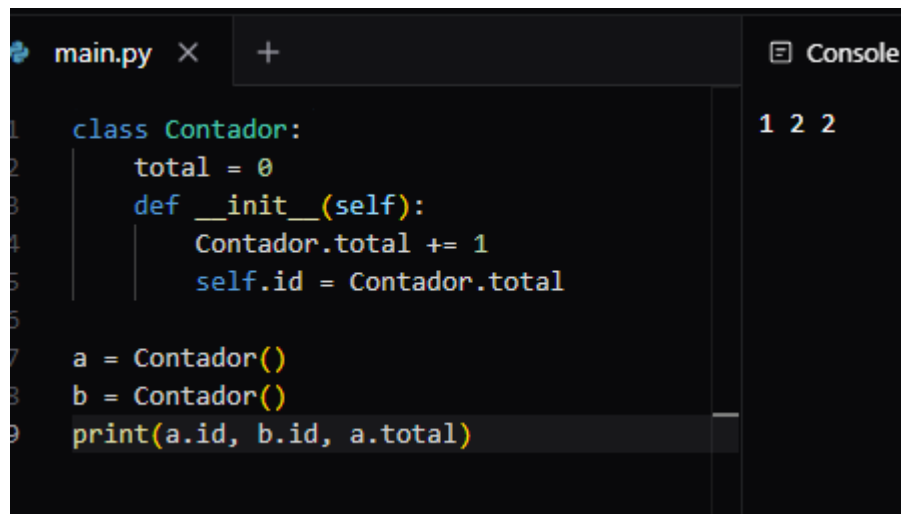
```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          class A { String nome = "A";
6              String getNome() { return nome; } }
7          class B extends A { String nome = "B";
8              String getNome() { return nome; } }
9
10         A x = new B();
11         System.out.println(x.nome + " " + x.getNome());
12     }
13 }
```

- (b) A decisão foi tomada na compilação ou na execução?
 - Para o atributo (`x.nome`): A decisão foi tomada em **compilação** (vinculação estática / *static binding*). Em Java, variáveis de instância sofrem sombreamento (*shadowing*), e o compilador escolhe o atributo com base no **tipo da referência** (A), e não no tipo real do objeto criado (B). Portanto, imprime "A".
 - Para o método (`x.getNome()`): A decisão foi tomada em **execução** (vinculação dinâmica / *dynamic dispatch* ou polimorfismo). Em Java, os métodos são polimórficos por padrão, logo a JVM determina qual método executar com base no **tipo do objeto real** (B) instanciado em memória. Portanto, chama o método de B, que retorna "B".

4. Python

- (a) Qual é a saída?

1 2 2



```
main.py x + Console
1 class Contador:
2     total = 0
3     def __init__(self):
4         Contador.total += 1
5         self.id = Contador.total
6
7 a = Contador()
8 b = Contador()
9 print(a.id, b.id, a.total)
```

1 2 2

- (b) A decisão foi tomada na compilação ou na execução?

A decisão e a resolução de atributos (tanto de classe quanto de instância) ocorrem inteiramente em **execução** (*runtime*). Como Python é uma linguagem dinamica e interpretada, a criação das classes, o incremento do atributo de classe total e a busca por atributos em a e b são avaliados sequencialmente à medida que o código é executado:

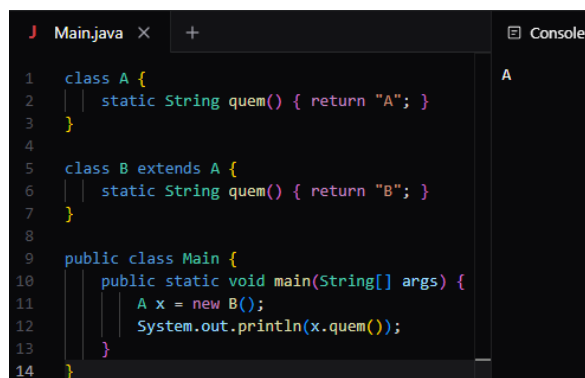
a = Contador() atribui id = 1 e incrementa total para 1.

b = Contador() atribui id = 2 e incrementa total para 2.

print imprime a.id (1), b.id (2) e a.total (que consulta o valor atualizado de total na classe, resultando em 2).

5. Java

- a) Qual é a saída?



```
Main.java x + Console
1 class A {
2     static String quem() { return "A"; }
3 }
4
5 class B extends A {
6     static String quem() { return "B"; }
7 }
8
9 public class Main {
10     public static void main(String[] args) {
11         A x = new B();
12         System.out.println(x.quem());
13     }
14 }
```

A

- b) A decisão foi tomada na compilação ou na execução?

A decisão é tomada na compilação. O método quem() é static, então não há polimorfismo: métodos estáticos não são sobrescritos (override), apenas ocultados (hiding). A chamada é resolvida pelo tipo estático (declarado) da variável, que é A, e não pelo tipo real do objeto (B). Na prática, x.quem() é compilado como A.quem().

6. Go

- a) Qual é a saída?



```
main.go x
1 package main
2
3 import "fmt"
4
5 type Animal struct{}
6
7 func (Animal) Som() string { return "... " }
8
9 func (a Animal) Falar() string {
10 |     return "faz " + a.Som()
11 | }
12
13 type Cao struct{ Animal }
14
15 func (Cao) Som() string { return "au" }
16
17 func main() {
18 |     fmt.Println(Cao{}.Falar(), Cao{}.Som())
19 | }
```

Console

faz ... au

- b) A decisão foi tomada na compilação ou na execução?

Decisão tomada na compilação. Em Go, a incorporação (embedding) de Animal em Cao não é herança.